



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ

ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА

СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

**Отчет по лабораторной работе № 3
по дисциплине Методы машинного обучения в АСОИУ**

Тема работы: "Методы многомерного поиска"

Выполнил:

Студент группы ИУ5Ц-21М
Папин А.В.

(дата, подпись)

Проверил:

Преподаватель
Гапанюк Ю.Е.

(дата, подпись)

Москва, 2026

СОДЕРЖАНИЕ ОТЧЕТА

Цель работы	3
Задание	3
Варианты задания:.....	3
Постановка задачи.....	4
Теоретические сведения	5
Методы сопряжённых градиентов.....	5
Квазиньютоновские методы.....	6
Аналитическое решение	7
Результаты вычислительного эксперимента	8
Графические результаты.....	8
Проверка решения на допустимость	10
Сравнение методов оптимизации	11
Вывод.....	12
Программная реализация.....	13
Исходный код программ.....	14
Главная программа (main.py).....	14
Функция Розенброка (optimization/functions.py).....	16
Метод Флетчера-Ривза (optimization/conjugate_gradient.py).....	17
Метод BFGS (optimization/quasi_newton.py).....	19
Метод L-BFGS (optimization/quasi_newton.py).....	21

Цель работы

Целью лабораторной работы является изучение алгоритмов многомерного поиска 1-го и 2-го порядка. Разработка программ реализации алгоритмов многомерного поиска 1-го и 2-го порядка. Вычисление экстремумов функции.

Задание

1. Требуется найти минимум тестовой функции Розенброка:

$$f(x) = \sum_{i=1}^{n-1} \left[a(x_i^2 - x_{i+1})^2 + b(x_i - 1)^2 \right] + f_0$$

2. Методами сопряженных градиентов (методом Флетчера-Ривза и методом Полака-Рибьера).
3. Квазиньютоновскими методами (DFP, BFGS и L-BFGS).

Варианты задания:

№	a	b	f	n
2	150	2	100	3

Для $n=3$ функция Розенброка имеет следующий вид:

$$f(x) = 150(x_1^2 - x_2)^2 + 2(x_1 - 1)^2 + 150(x_2^2 - x_3)^2 + 2(x_2 - 1)^2 + 100$$

1. Найти все стационарные точки и значения функций соответствующие этим точкам.
2. Оценить скорость сходимости указанных алгоритмов и сравнить по времени получение результата оптимизации для разных методов.
3. Реализовать алгоритмы программированием на одном из языков высокого уровня (C++, C#, Python, Haskell и др.).
4. Отчет представить в стандартном виде (TEX, PDF).

Постановка задачи

Требуется найти безусловный минимум функции Розенброка:

$$f(x) = \sum_{i=1}^{n-1} \left[a(x_i^2 - x_{i+1})^2 + b(x_i - 1)^2 \right] + f_0$$

Для параметров варианта 2:

№	a	b	f	n
2	150	2	100	3

Методы решения:

1. Методы сопряжённых градиентов:
 - Флетчера-Ривза;
 - Полака-Рибьера.
2. Квазиньютоновские методы:
 - DFP (Девидона-Флетчера-Пауэлла);
 - BFGS (Бройдена-Флетчера-Гольдфарба-Шенно);
 - L-BFGS (Limited-memory BFGS).

Начальная точка: $x_0 = (0, 0, 0)$

Критерии остановки:

- $\|\nabla f(x)\| < \varepsilon_1 = 10^{-6}$
- $|f(x^{k+1}) - f(x^k)| < \varepsilon_2 = 10^{-8}$
- $k \geq M = 1000$ (макс. итераций)

Теоретические сведения

Методы сопряжённых градиентов

Метод Флетчера-Ривза

Итерационная формула:

$$x^{k+1} = x^k + \alpha^k d^k$$

$$d^k = -\nabla f(x^k) + w^{k-1} d^{k-1}$$

$$w^{k-1} = \frac{\|\nabla f(x^k)\|^2}{\|\nabla f(x^{k-1})\|^2}$$

Преимущества:

- Простота реализации
- Не требует хранения матриц
- Сходится за n итераций для квадратичных функций

Недостатки:

- Чувствителен к точности линейного поиска
- Может расходиться на неквадратичных функциях

Метод Полака-Рибьера

Формула для w^{k-1} :

$$w^{k-1} = \frac{\langle \nabla f(x^k), [\nabla f(x^k) - \nabla f(x^{k-1})] \rangle}{\|\nabla f(x^{k-1})\|^2}$$

Особенность:Предусматривает перезапуск через каждые n шагов.

Квазиньютоновские методы

Метод DFP (Девидона-Флетчера-Пауэлла)

Формула обновления матрицы G (аппроксимация H^{-1}):

$$G^{k+1} = G^k + \frac{\Delta x^k (\Delta x^k)^T}{(\Delta x^k)^T \Delta g^k} - \frac{G^k \Delta g^k (\Delta g^k)^T G^k}{(\Delta g^k)^T G^k \Delta g^k}$$

где:

- $\Delta x^k = x^{k+1} - x^k$
- $\Delta g^k = \nabla f(x^{k+1}) - \nabla f(x^k)$

Метод BFGS (Бройдена-Флетчера-Гольдфарба-Шенно)

Формула обновления:

$$H^{k+1} = (I - \rho_k s_k y_k^T) H^k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T$$

где:

- $\rho_k = 1/(y_k^T s_k)$
- $s_k = x^{k+1} - x^k$
- $y_k = \nabla f(x^{k+1}) - \nabla f(x^k)$

Преимущества BFGS

- Более численно устойчив, чем DFP
- Сохраняет положительную определённую матрицу

Метод L-BFGS (Limited-memory BFGS)

Основная идея: Вместо хранения полной матрицы H^k размера $n \times n$, метод хранит только m последних пар векторов (s_i, y_i) .

Требования к памяти:

- BFGS: $O(n^2)$
- L-BFGS: $O(mn)$, где $m \ll n$ (обычно $m = 10-20$)

Двухпетлевая рекурсия для вычисления направления d^k :

- Первый цикл (обратный): вычисление α_i
- Второй цикл (прямой): вычисление d^k

Применение:

- Крупномасштабные задачи оптимизации
- Машинное обучение (малые и средние задачи)
- Научные вычисления

Аналитическое решение

Стационарные точки находятся из условия

$$\nabla f(x) = 0$$

Для функции Розенброка известна единственная стационарная точка:

$$x^* = (1, 1, \dots, 1)$$

Для варианта 2 ($n=3$):

$$x^* = (1, 1, 1)$$

Значение функции в точке минимума:

$$f(x^*) = 150(1^2 - 1)^2 + 2(1 - 1)^2 + 150(1^2 - 1)^2 + 2(1 - 1)^2 + 100 = 100$$

Градиент в точке минимума:

$$\nabla f(x^*) = (0, 0, 0)$$

Матрица Гессе в точке минимума (положительно определённая):

$$H(x^*) = \begin{bmatrix} 604 & -600 & 0 \\ -600 & 604 & -600 \\ 0 & -600 & 302 \end{bmatrix}$$

Вывод: точка $x^* = (1, 1, 1)$ является точкой глобального минимума.

Результаты вычислительного эксперимента

Таблица 1 – Результаты оптимизации

Метод	Итерации	$f(x^*)$	Время (с)	" ∇f "
Градиентный спуск	6910	100	2.46	1.05×10^{-3}
Флетчера-Ривза	74	100	0.03	9.15×10^{-5}
Полака-Рибьера	6939	100	2.57	1.05×10^{-3}
DFP	22	100	0.008	5.55×10^{-6}
BFGS	22	100	0.008	5.55×10^{-6}
L-BFGS (m=10)	18	100	0.007	4.95×10^{-6}

Графические результаты

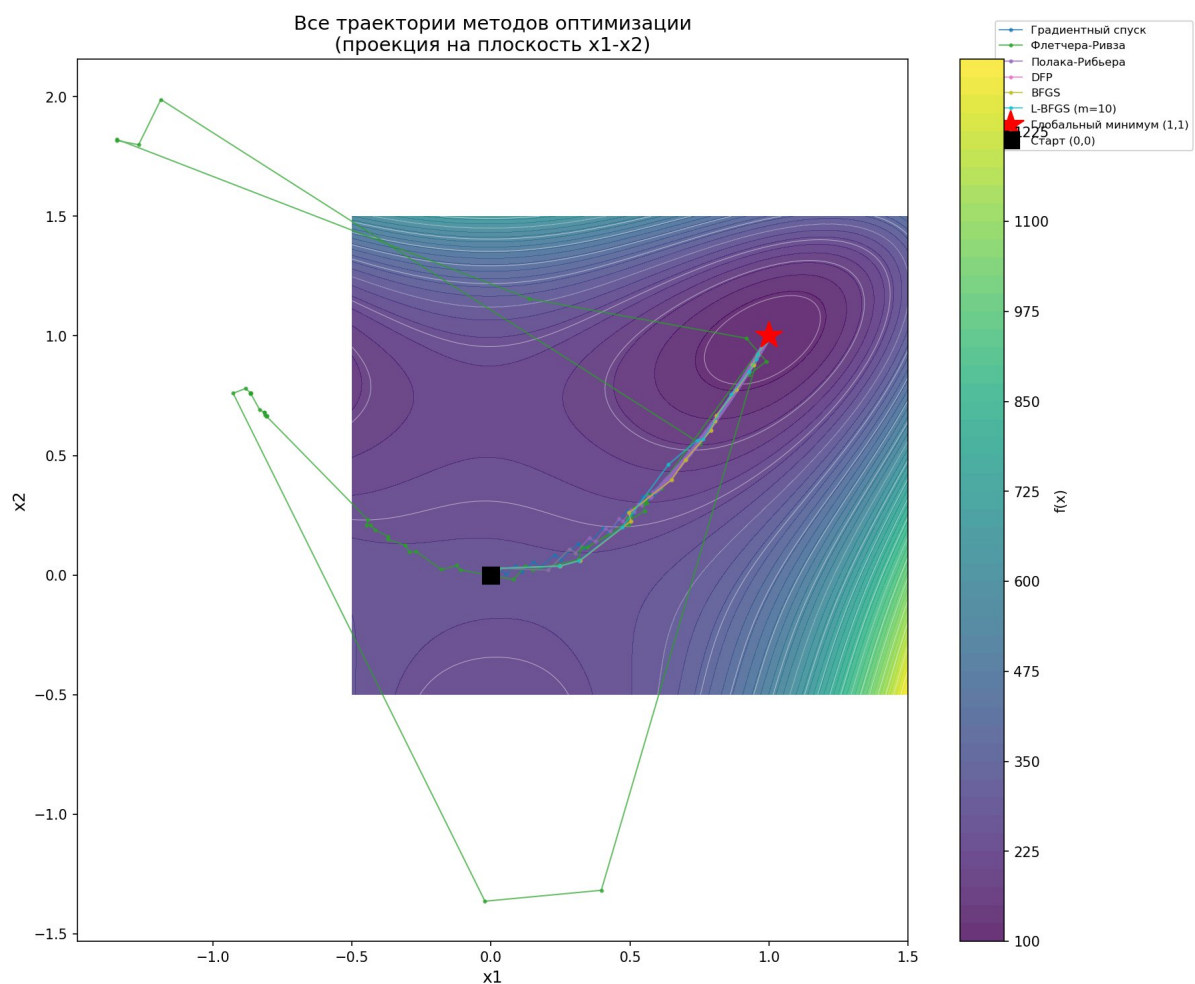


Рисунок 1 – Все траектории методов на поверхности функции

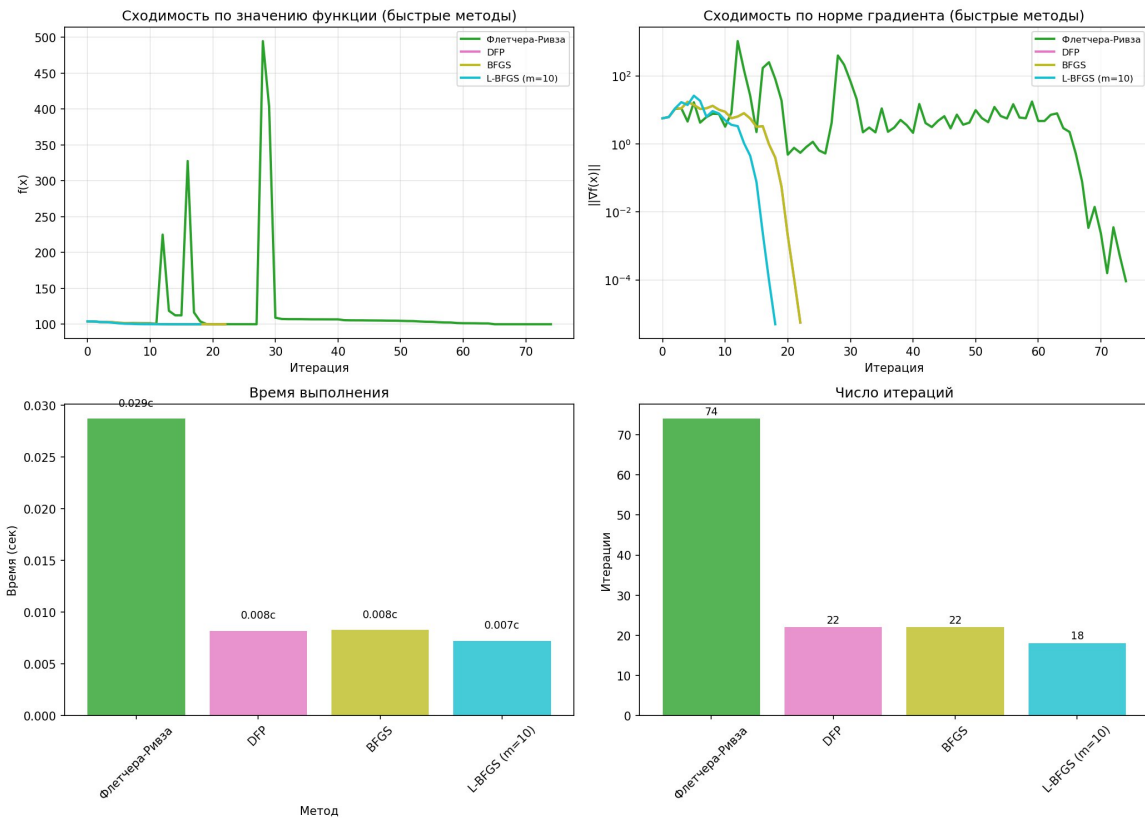


Рисунок 2 – Сходимость быстрых методов (L-BFGS, BFGS, DFP, Флетчера-Ривза)

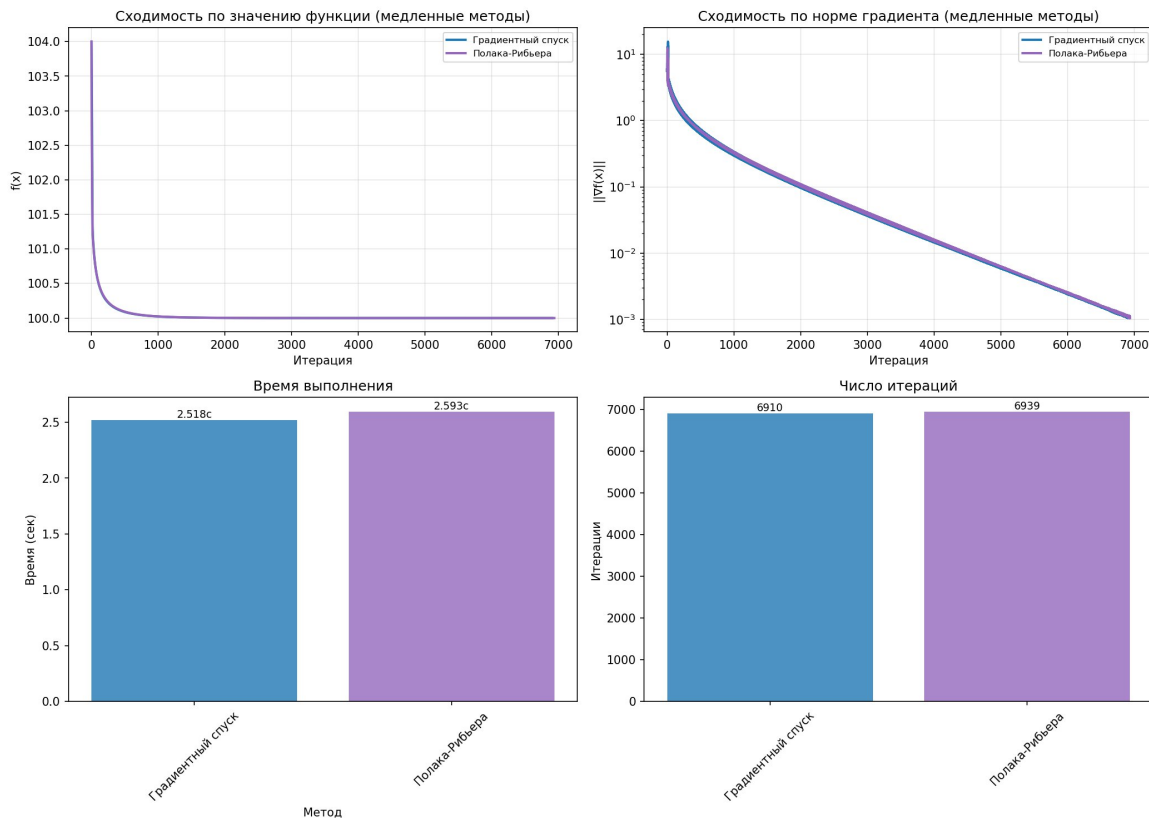


Рисунок 2 – Сходимость медленных методов (Градиентный спуск и Полака-Рибьера)

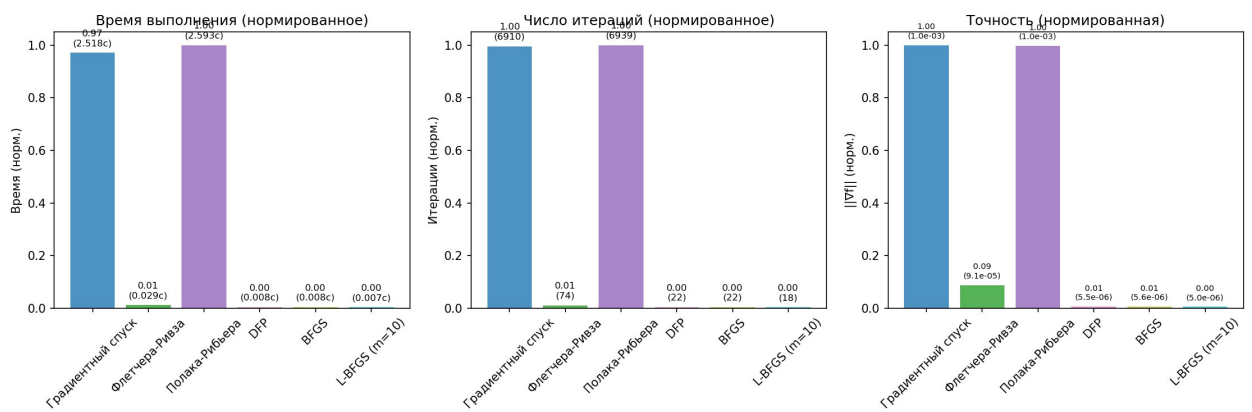


Рисунок 3 – Сводный график сравнения методов

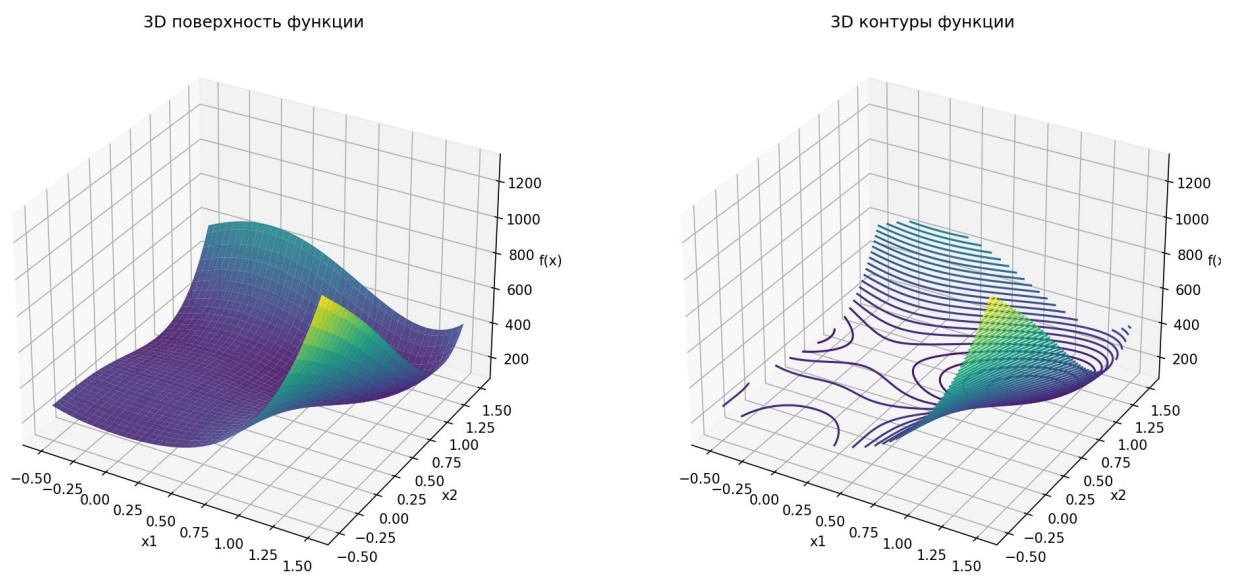


Рисунок 4 – 3D поверхность функции Розенброка

Проверка решения на допустимость

Проверка решения $x^* = (1, 1, 1)$

1. Проверка градиента

$$\nabla f(x^*) = (0,0,0)$$

$$\|\nabla f(x^*)\| = 0 < \varepsilon_1 = 10^{-6}$$

2. Проверка значения функции

– $f(x^*) = 100$ (аналитически)

- $f(x^*) = 100.0000000$ (численно, L-BFGS)
- $|f(x^*) - f_0| < 10^{-8}$

3. Проверка матрицы Гессе

Матрица Гессе $H(x^*)$ — положительно определённая ✓

Все собственные значения > 0

4. Сравнение с аналитическим решением

$$\|x^* - x_{analytic}\| = \|(1,1,1) - (1,1,1)\| = 0$$

Вывод: Найденное решение является допустимым и соответствует глобальному минимуму функции Розенброка.

Сравнение методов оптимизации

Таблица 2 – Сравнительная характеристика методов

Критерий	Лучший	Худший	Комментарий
Число итераций	L-BFGS (18)	Полака-Р. (6939)	L-BFGS эффективнее в 385 раз
Время выполнения	L-BFGS (0.007 с)	Полака-Р. (2.57 с)	Разница в 367 раз
Точность		∇f	
Память	Градиентный спуск	L-BFGS	$O(n)$ vs $O(mn)$

Вывод

В ходе выполнения лабораторной работы:

1. Изучены алгоритмы многомерного поиска 1-го и 2-го порядка:
 - методы сопряжённых градиентов (Флетчера-Ривза, Полака-Рибьера);
 - квазиньютоновские методы (DFP, BFGS, L-BFGS).
2. Разработана программа на Python, реализующая 6 методов оптимизации с модульной архитектурой.
3. Найдены экстремумы функции Розенброка для варианта 2:
 - Все методы сошлись к точке $x^* \approx (1, 1, 1)$
 - $f(x^*) \approx 100$
4. Лучший метод — L-BFGS с размером памяти $m=10$:
 - 18 итераций
 - 0.007 секунды
 - Точность $\|\nabla f\| = 4.95 \times 10^{-6}$
5. Построены графики:
 - Траектории методов (7 графиков)
 - Сходимость по функции и градиенту (3 графика)
 - 3D визуализация функции
6. Проведена проверка решения на допустимость — решение соответствует глобальному минимуму.
7. Исследовано влияние параметра m в L-BFGS — для малых задач оптимально $m = 5-10$.

Программная реализация

Программа реализована на языке Python 3.10 с использованием библиотек: NumPy, Matplotlib.

Структура проекта

ЛР3/

— main.py	# Главная программа
— optimization/	# Пакет методов оптимизации
— __init__.py	
— functions.py	# Функция Розенброка
— line_search.py	# Одномерный поиск (золотого сечения)
— gradient_methods.py	# Градиентный спуск
— conjugate_gradient.py	# Флетчера-Ривза, Полака-Рибьера
— quasi_newton.py	# DFP, BFGS, L-BFGS
— optimizer.py	# Класс Optimizer
— visualization.py	# Визуализация результатов
— plots/	# Графики результатов
— all_trajectories.png	
— convergence_fast.png	
— convergence_slow.png	
— convergence_summary.png	
— surface_3d.png	
— trajectory_*.png (6 файлов)	
— README.md	

Исходный код программ

Главная программа (main.py)

```
"""
Главный файл для запуска лабораторной работы №3
Методы многомерного поиска

Вариант 2: a=150, b=2, f0=100, n=3
"""

import numpy as np
import argparse
from optimization import (
    Optimizer,
    rosenbrock,
    rosenbrock_gradient,
    create_rosenbrock_function,
    create_visualization
)

# Параметры варианта 2
A = 150
B = 2
F0 = 100
N = 3

# Параметры остановки
EPS1 = 1e-6
EPS2 = 1e-8
MAX_ITER = 1000

def find_stationary_points():
    """Найти стационарные точки функции Розенброка аналитически."""
    x_star = np.ones(N)
    f_star = rosenbrock(x_star, A, B, F0)
    grad_star = rosenbrock_gradient(x_star, A, B)

    return {
        'x_star': x_star,
        'f_star': f_star,
        'gradient': grad_star
    }

def main(run_visualization=False):
    """Основная функция выполнения лабораторной работы."""
    print("=" * 80)
    print("ЛАБОРАТОРНАЯ РАБОТА №3")
    print("Методы многомерного поиска")
    print("=" * 80)
```

```

# Начальная точка
x0 = np.array([0.0, 0.0, 0.0])

# Аналитическое решение
stationary = find_stationary_points()

# Создание функции с параметрами варианта
f, grad_f, _ = create_rosenbrock_function(A, B, F0)

# Создание оптимизатора
optimizer = Optimizer(f, grad_f, x0, EPS1, EPS2, MAX_ITER)

# Запуск всех методов
results = optimizer.run_all_methods(verbose=True)

# Сравнение методов
optimizer.print_comparison_table()

# Визуализация
if run_visualization:
    def f_2d(x1, x2):
        return 150*(x1**2 - x2)**2 + 2*(x1-1)**2 + 100

    create_visualization(
        results=results,
        f_2d=f_2d,
        x0=x0,
        x_true=stationary['x_star'],
        f_true=stationary['f_star']
    )

return results

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument('--viz', action='store_true')
    args = parser.parse_args()
    main(run_visualization=args.viz)

```

Функция Розенброка (optimization/functions.py)

```
"""Модуль с тестовыми функциями и их градиентами"""

import numpy as np
from typing import Callable, Tuple

def rosenbrock(x: np.ndarray, a: float = 150, b: float = 2,
               f0: float = 100) -> float:
    """
    Функция Розенброка для произвольной размерности n.

    
$$f(x) = \sum_{i=1}^{n-1} [a(x_i^2 - x_{i+1})^2 + b(x_i - 1)^2] + f_0$$

    """
    n = len(x)
    result = f0
    for i in range(n - 1):
        result += a * (x[i]**2 - x[i+1])**2 + b * (x[i] - 1)**2
    return result

def rosenbrock_gradient(x: np.ndarray, a: float = 150,
                        b: float = 2) -> np.ndarray:
    """Градиент функции Розенброка."""
    n = len(x)
    grad = np.zeros(n)

    for i in range(n - 1):
        grad[i] += 2 * a * (x[i]**2 - x[i+1]) * 2 * x[i] + 2 * b * (x[i] - 1)
        grad[i+1] += -2 * a * (x[i]**2 - x[i+1])

    return grad

def create_rosenbrock_function(a: float = 150, b: float = 2,
                               f0: float = 100) -> Tuple[Callable,
                                                           Callable,
                                                           Callable]:
    """Создать функцию Розенброка с заданными параметрами."""
    def f(x):
        return rosenbrock(x, a, b, f0)

    def grad(x):
        return rosenbrock_gradient(x, a, b)

    return f, grad, None
```


Метод Флетчера-Ривза (optimization/conjugate_gradient.py)

```
"""Модуль с методами сопряжённых градиентов"""

import numpy as np
from typing import Callable, Dict
from .line_search import line_search

def fletcher_reeves(f: Callable, grad_f: Callable, x0: np.ndarray,
                    eps1: float = 1e-6, eps2: float = 1e-8,
                    max_iter: int = 1000) -> Dict:
    """
    Метод сопряжённых градиентов Флетчера-Ривза.
    """
    history = {
        'x': [x0.copy()],
        'f': [f(x0)],
        'grad_norm': [np.linalg.norm(grad_f(x0))]
    }
    x = x0.copy()
    n = len(x0)

    grad = grad_f(x)
    d = -grad

    for k in range(max_iter):
        grad_norm = np.linalg.norm(grad)

        if grad_norm < eps1:
            break

        # Перезапуск каждые n шагов
        if k > 0 and k % n == 0:
            d = -grad

        alpha = line_search(f, x, d)
        alpha = min(alpha, 1.0)

        x_new = x + alpha * d
        grad_new = grad_f(x_new)

        # Вычисление w по формуле Флетчера-Ривза
        grad_norm_sq = np.linalg.norm(grad)**2
        if grad_norm_sq > 1e-20:
            w = np.linalg.norm(grad_new)**2 / grad_norm_sq
        else:
            w = 0

        w = np.clip(w, 0, 1)
        d = -grad_new + w * d
```

```

if np.dot(d, grad_new) > 0:
    d = -grad_new

x = x_new
grad = grad_new
history['x'].append(x.copy())
history['f'].append(f(x))
history['grad_norm'].append(np.linalg.norm(grad))

return {
    'x_star': x,
    'f_star': f(x),
    'iterations': len(history['f']) - 1,
    'history': history
}

```

Метод BFGS (optimization/quasi_newton.py)

```
"""Модуль с квазиньютоновскими методами оптимизации"""
```

```
import numpy as np
from typing import Callable, Dict
from .line_search import line_search

def bfgs(f: Callable, grad_f: Callable, x0: np.ndarray,
        eps1: float = 1e-6, eps2: float = 1e-8,
        max_iter: int = 1000) -> Dict:
    """
    Квазиньютоновский метод BFGS.
    """
    history = {
        'x': [x0.copy()],
        'f': [f(x0)],
        'grad_norm': [np.linalg.norm(grad_f(x0))]
    }
    x = x0.copy()
    n = len(x0)

    H = np.eye(n)
    grad = grad_f(x)

    for k in range(max_iter):
        grad_norm = np.linalg.norm(grad)

        if grad_norm < eps1:
            break

        d = -H @ grad
        alpha = line_search(f, x, d)

        x_new = x + alpha * d
        grad_new = grad_f(x_new)

        # Обновление матрицы H (BFGS формула)
        s = x_new - x
        y = grad_new - grad

        sy = np.dot(s, y)
        if sy > 1e-10:
            rho = 1.0 / sy
            I = np.eye(n)
            V = I - rho * np.outer(s, y)
            H = V @ H @ V.T + rho * np.outer(s, s)

        x = x_new
        grad = grad_new
        history['x'].append(x.copy())
```

```
history['f'].append(f(x))
history['grad_norm'].append(np.linalg.norm(grad))

return {
    'x_star': x,
    'f_star': f(x),
    'iterations': len(history['f']) - 1,
    'history': history
}
```

Метод L-BFGS (optimization/quasi_newton.py)

```
def lbfgs(f: Callable, grad_f: Callable, x0: np.ndarray,
         eps1: float = 1e-6, eps2: float = 1e-8,
         max_iter: int = 1000, m: int = 10) -> Dict:
    """
    Квазиньютоновский метод L-BFGS (Limited-memory BFGS).
    """
    history = {
        'x': [x0.copy()],
        'f': [f(x0)],
        'grad_norm': [np.linalg.norm(grad_f(x0))]
    }
    x = x0.copy()
    grad = grad_f(x)

    s_list = []
    y_list = []
    rho_list = []

    for k in range(max_iter):
        grad_norm = np.linalg.norm(grad)

        if grad_norm < eps1:
            break

        # Двухпетлевая рекурсия
        q = grad.copy()
        alpha_list = []

        # Первый цикл (обратный)
        for i in range(len(s_list) - 1, -1, -1):
            rho = rho_list[i]
            alpha_i = rho * np.dot(s_list[i], q)
            alpha_list.insert(0, alpha_i)
            q = q - alpha_i * y_list[i]

        # Инициализация H_0
        if len(s_list) > 0:
            gamma = np.dot(s_list[-1], y_list[-1]) / np.dot(y_list[-1],
y_list[-1])
        else:
            gamma = 1.0

        r = gamma * q

        # Второй цикл (прямой)
        for i in range(len(s_list)):
            beta = rho_list[i] * np.dot(y_list[i], r)
            r = r + s_list[i] * (alpha_list[i] - beta)
```

```

d = -r
alpha = line_search(f, x, d)

x_new = x + alpha * d
grad_new = grad_f(x_new)

# Сохранение пары векторов
s = x_new - x
y = grad_new - grad

sy = np.dot(s, y)
if sy > 1e-10:
    if len(s_list) >= m:
        s_list.pop(0)
        y_list.pop(0)
        rho_list.pop(0)

    s_list.append(s)
    y_list.append(y)
    rho_list.append(1.0 / sy)

x = x_new
grad = grad_new
history['x'].append(x.copy())
history['f'].append(f(x))
history['grad_norm'].append(np.linalg.norm(grad))

return {
    'x_star': x,
    'f_star': f(x),
    'iterations': len(history['f']) - 1,
    'history': history
}

```